

# A Learning-Assisted Bi-Population Evolutionary Algorithm for Distributed Flexible Job-Shop Scheduling With Maintenance Decisions

Qi Yan<sup>✉</sup>, Hongfeng Wang<sup>✉</sup>, and Shengxiang Yang<sup>✉</sup>, *Senior Member, IEEE*

**Abstract**—In the post-pandemic era, more manufacturers have expedited the shift from centralized to distributed manufacturing to enhance supply chain resilience. Along with this, the distributed shop floor scheduling problem has attracted much attention from academia, one of which is the distributed flexible job-shop scheduling problem (DFJSP). Nonetheless, the majority of research on DFJSPs overlooks crucial real-world necessities, such as multiobjective decision making and preventive maintenance (PM). Thus, this article suggests a multiobjective DFJSP with PM (DFJSP/PM) as a new variant of the DFJSP. The aim is to achieve a tradeoff between production and maintenance to minimize the makespan, maintenance cost, and energy consumption. To this end, we establish a mathematical model and then customize a learning-assisted bi-population evolutionary algorithm (LBPEA) to solve it. In LBPEA, a novel encoding mechanism is proposed to initialize the population randomly. Then, a neighborhood search heuristic is designed to enhance the population's quality. To balance the convergence and diversity of the population, a bi-population evolution idea is introduced during the environmental selection. Besides, a two-stage local search (LS) process is adaptively triggered to balance the allocation of computational resources between exploration and exploitation. At the first stage, a reinforcement learning mechanism is employed to intelligently select LS operators to adjust either the operations' sequence or assignment to different factories and machines, while the second stage is to adjust the number and placement of maintenance decisions. Experimental results show that LBPEA has excellent performance in terms of convergence and diversity when solving the proposed multiobjective DFJSP/PM.

**Index Terms**—Distributed manufacturing, joint optimization, multiobjective evolutionary algorithm (MOEA), preventive maintenance (PM), reinforcement learning (RL).

Manuscript received 21 January 2024; revised 5 April 2024; accepted 8 May 2024. Date of publication 13 May 2024; date of current version 8 October 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFF0901300; in part by the NSFC Key Supported Project of the Major Research Plan under Grant 92267206; in part by the National Natural Science Foundation of China under Grant 62173076 and Grant 72271048; and in part by the China Scholarship Council under Grant 202206080076. This article was approved by Associate Editor L. Wang. (Corresponding authors: Hongfeng Wang; Shengxiang Yang.)

Qi Yan and Hongfeng Wang are with the College of Information Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: yanqz@stumail.neu.edu.cn; hfwang@mail.neu.edu.cn).

Shengxiang Yang is with the School of Computer Science and Informatics, De Montfort University, LE1 9BH Leicester, U.K. (e-mail: syang@dmu.ac.uk).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TEVC.2024.3400043>, provided by the authors.

Digital Object Identifier 10.1109/TEVC.2024.3400043

## I. INTRODUCTION

**D**URING the COVID-19 pandemic, the global supply chain suffered severe disruptions, with many manufacturers facing raw material shortages, production disruptions, and transportation uncertainties [1]. As a result, the manufacturing industry has hastened its move from centralized to distributed manufacturing. One key feature of distributed manufacturing is that it disperses production capabilities across multiple locations [2]. On the one hand, it can improve production efficiency and enhance supply chain resilience, enabling better responses to rapidly changing market demands [3]. On the other hand, it can reduce transportation and carbon emissions, thereby promoting sustainable manufacturing [4].

To advance the development of distributed manufacturing, distributed shop floor scheduling problems have garnered increasing attention in recent years [5], one of which is the distributed flexible job-shop scheduling problem (DFJSP) [6], [7]. Compared with the flexible job-shop scheduling problem (FJSP) in centralized manufacturing [8], the DFJSP introduces an additional subproblem, i.e., the distribution of orders among factories in a distributed production network. An efficient assignment scheme can achieve a balanced workload across the entire network, thereby enhancing efficiency in distributed manufacturing.

Despite previous efforts on the DFJSP, limitations still exist. Most variants of the DFJSP assume that the machine is always available, overlooking the imperative need for preventive maintenance (PM) in real-world scenarios. The significance of PM has been extensively demonstrated in centralized manufacturing scenarios, such as the single machine [9], parallel machines [10], flow shops [11], and job shops [12]. Therefore, advancing research on the DFJSP with PM (DFJSP/PM) holds considerable scientific and practical significance. Considering that existing research on the DFJSP/PM primarily focuses on single-objective optimization, particularly in minimizing the makespan [13], [14], [15], [16], this article presents a multiobjective DFJSP/PM. Our aim is to strike a balance between production and PM to minimize the makespan, maintenance cost, and total energy consumption. This not only aligns more closely with real-world requirements but also fills the gap in research concerning multiobjective decision making of production and maintenance within the domain of the DFJSP.

Developing an efficient multiobjective evolutionary algorithm (MOEA) tailored to the characteristics of the

proposed multiobjective DFJSP/PM is a significant challenge to be addressed in this article. The mainstream algorithm frameworks for solving multiobjective shop floor scheduling problems are dominance-based MOEAs [6], [17] and decomposition-based MOEAs [18], [19]. Both types of MOEAs have their own advantages, but most of them are based on the “convergence first and diversity second” principle. In fact, convergence and diversity are equally important [20]. For the tradeoff, we integrate the two algorithm frameworks during the environmental selection, borrowing the bi-population evolution idea from [21] and [22]. When the evolutionary process becomes trapped in local optima, implementing local search (LS) strategies may find more promising solutions by improving the neighborhood structure of solutions, thereby avoiding premature convergence of the algorithm. Consequently, we design a learning-assisted adaptive LS strategy, and the frequency of LS is adaptively adjusted based on variations in the Pareto front, aiming to balance the allocation of computational resources between exploration and exploitation. Furthermore, there are two stages to adjust production and maintenance decisions, respectively. At the first stage, a reinforcement learning (RL) algorithm is employed to intelligently select respective LS operators based on the positional relationship between the nondominated solutions and ideal point, which can enhance the algorithm’s overall performance without increasing its complexity. At the second stage, a maintenance adjustment strategy is proposed to effectively tackle the huge maintenance decisions.

On the basis of the above statement, the primary contributions of this article are summarized below.

- 1) We establish a mixed-integer programming model for the proposed multiobjective DFJSP/PM.
- 2) We tailor a bi-population MOEA that can balance convergence and diversity to efficiently solve the model.
- 3) We design an RL-assisted LS selection strategy, in which the definitions of state, action, and reward can be further extended and applied to higher-dimensional MOEAs.

The remainder of this article is organized as follows. Section II presents related works. Section III describes the studied multiobjective DFJSP/PM and establishes the corresponding mathematical model, followed by the developed learning-assisted bi-population evolutionary algorithm (LBPEA) in Section IV. Next, the performance of LBPEA is tested on a series of instances in Section V. Finally, Section VI summarizes the contributions and gives some potential research directions.

## II. LITERATURE REVIEW

This section reviews and summarizes the relevant studies of the DFJSP/PM and applications of RL in production scheduling to highlight the research gaps between this article and related works.

### A. Relevant Studies of the DFJSP/PM

Previous research on the DFJSP has depicted a variety of constraints as well as optimization objectives and developed optimization approaches accordingly [23], [24], [25].

TABLE I  
REPRESENTATIVE RESEARCH ON THE DFJSP/PM

| Reference | Role of PMs       | Objective | Approach    |
|-----------|-------------------|-----------|-------------|
| [13]      | Constraint        | Makespan  | Modified GA |
| [14]      | Decision variable | Makespan  | Modified GA |
| [15]      | Decision variable | Makespan  | Modified GA |
| [16]      | Decision variable | Makespan  | Modified GA |

However, only very sparse studies have considered machine unavailability due to PM in the DFJSP. Some representative research on the DFJSP/PM is presented in Table I, where GA denotes the genetic algorithm.

In terms of the integration way of the DFJSP and PM, Chan et al. [13] treated the predefined PM strategy as fixed constraints in the distributed production process, which fails to fully consider the mutual influence between PM and production scheduling. In contrast, [14], [15], and [16] made joint decisions of the DFJSP and PM, and developed respective GAs to effectively solve the studied DFJSP/PM. Due to significantly expanding the decision space, this integration approach has the potential to explore better solutions. However, current research on DFJSP/PM only focuses on minimizing the makespan using customized evolutionary algorithms with specific chromosome representations.

In real-world manufacturing scenarios, it is necessary to consider multiple indicators, such as time, cost, and energy consumption. Therefore, multiobjective optimization has become an inevitable trend in the research of the DFJSP/PM. In this article, we propose a multiobjective DFJSP/PM aimed at simultaneously optimizing the makespan, maintenance cost, and energy consumption, in which PM is treated as a virtual job involved in the decision-making process of distributed flexible job-shop scheduling. The studied multiobjective integrated decisions of distributed flexible job-shop scheduling and PM will cover a research gap within the domain of the DFJSP as well as the integrated optimization of production scheduling and PM planning.

### B. Applications of RL in Production Scheduling

RL is a machine learning method that enables the agent to make optimal decisions by learning through interaction with the environment. It has been increasingly used in the field of production scheduling in recent years [26]. On the one hand, RL can be treated as an independent optimization approach to solve production scheduling problems. Common RL methods include, value-based RL, e.g.,  $Q$ -learning [9] and SARSA [27], policy-based RL, e.g., proximal policy optimization [28], and combinations of the above methods, e.g., asynchronous advantage actor-critic [29] and deep deterministic policy gradient [30]. On the other hand, RL can assist evolutionary algorithms to learn either key parameters [31] or search strategies [32] to improve the solution performance.

This work comes within the second category of RL applications discussed above. We first customize a knowledge-driven bi-population MOEA for the proposed multiobjective DFJSP/PM and then introduce the  $Q$ -learning algorithm to help select LS strategies in the MOEA. Previous RL designs

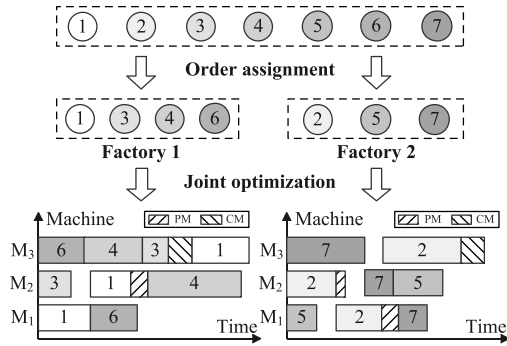


Fig. 1. Illustration of the proposed DFJSP/PM.

for multiobjective optimization problems have mostly used metrics, such as convergence and diversity [19] to measure the state and reward, while this article proposes a new metric aimed at encouraging the solution to move toward the dimension furthest from the ideal point.

### III. PROBLEM DESCRIPTION AND MODEL FORMULATION

#### A. Problem Description

A total of  $N$  orders are given to  $F$  geographically distributed factories for collaborative manufacturing. All factories are homogeneous, and each factory can be abstracted as a flexible job shop with  $M$  machines. Each order is completed in multiple steps, each of which can be processed by multiple available machines with respective normal processing times. Due to machine deterioration, the actual processing time of an operation is usually longer than its normal processing time, as defined in [33]. To alleviate the above deteriorating effects, PM is introduced to improve the running condition of aging machines. In this article, PM strategies are not predetermined, but are objective-driven and need to be decided in parallel with the production process. Considering the scenario where untimely PM may result in machine failures, a corrective maintenance (CM) strategy will be triggered once the machine's age exceeds a given threshold  $T$ . The duration of PM varies across the machine's age. As the machine deterioration becomes more severe, the time required to perform a PM will increase, but it must not exceed the duration for CM. An illustration of the proposed DFJSP/PM is shown in Fig. 1.

Regarding the optimization objectives, the first one is to minimize the maximum completion time of all factories (i.e., makespan), which is frequently used in related production scheduling problems. The second objective is to minimize the total maintenance cost, aiming to tradeoff PM and CM. The last one is to minimize the total energy consumption during production, standby and maintenance periods for energy-efficient scheduling. Overall, the aim of the proposed DFJSP/PM is to determine proper factory assignment (FA), machine selection (MS) for each operation, joint sequence of operations and PM on each machine, and ultimately to obtain high-quality nondominated solutions that can minimize the makespan, total maintenance cost and total energy consumption.

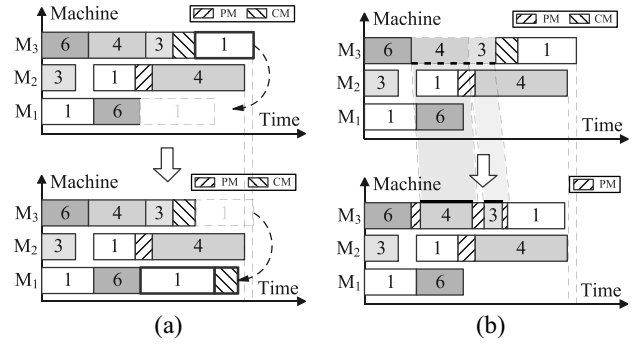


Fig. 2. Illustration of conflicts among objectives. (a) Scenario 1. (b) Scenario 2.

#### B. Conflicts Among Objectives

Prior to modeling, the analysis of the conflicting nature of the optimization objectives is important, which can ensure the soundness of the proposed multiobjective DFJSP/PM.

First, there is not necessarily a positive correlation between the makespan and total energy consumption. The makespan reflects the completion time of the last operation in all factories, while the total energy consumption covers the energy consumption of actual production, machine idling, and maintenance. As a simple example in Fig. 2(a), due to the flexibility of the machine, when the last operation is assigned to another machine with a longer processing time, the makespan may become shorter, but the longer processing time means an increase in energy consumption and may even trigger the CM mechanism and increase the total maintenance cost.

Second, an increase in the number of PMs may result in higher total maintenance cost, but may also improve the makespan and total energy consumption. This is because the execution of PMs can improve the machine deterioration and consequently reduce the actual production time. When the reduction in production time is greater than the increase in maintenance time, the total energy consumption can be reduced since the energy consumption of production is more than that of maintenance. If this scenario occurs on the machine with the longest completion time, the makespan can also be improved, as shown in Fig. 2(b).

The above description reflects the conflicts among the three optimization objectives only through obvious examples. Indeed, in the proposed DFJSP/PM, there are a high amount of implicit conflicts, which can be intuitively seen from the illustrative example in Section V-E.

#### C. Proposed Model

Some modeling assumptions are given and then the symbols to be used in the model are defined.

- 1) The setup time before the start of each operation is ignored.
- 2) The buffer space between different machines is large enough.
- 3) The transportation time of the operations on different machines is negligible.
- 4) The implementation of CM can avoid machine failures.

$f$ : Factory index,  $f = 1, \dots, F$ .  
 $m, m'$ : Machine index,  $m, m' = 1, \dots, M$ .  
 $i, g$ : Order index,  $i, g = 1, \dots, N$ .  
 $J_i$ : Operation set of order  $i$ .  
 $j, h$ : Operation index of order  $i$   $\forall j, h \in J_i$ .  
 $k, k'$ : Sequential index of operations of the same machine.  
 $O_{ij}$ : Operation  $j$  of order  $i$ .  
 $E_{ij}$ : Earliest start time of  $O_{ij}$ ,  $E_{ij} \geq 0$ .  
 $P_{ijm}$ : Normal processing time of  $O_{ij}$  on machine  $m$ .  
 $d$ : Linear deterioration rate.  
 $a_{ij}$ : Machine's age prior to  $O_{ij}$ ,  $a_{ij} \geq 0$ .  
 $P'_{ij}$ : Actual processing time of  $O_{ij}$ .  
 $I_{ij}$ : Idle time prior to  $O_{ij}$ ,  $I_{ij} \geq 0$ .  
 $t_{CM}$ : Duration of a CM.  
 $t_{ij}$ : Duration of the PM that is performed after  $O_{ij}$ ,  $t_{ij} \geq 0$ .  
 $T$ : Threshold for triggering the CM.  
 $L$ : Large positive number.  
 $c$ : Maintenance cost per unit time.  
 $\mu_1, \mu_2, \mu_3$ : Energy consumption per unit time in production, maintenance, and standby mode.  
 $\eta_{ij}$ : 1 if PM is performed after  $O_{ij}$  and 0 otherwise.  
 $q_{ij}$ : 1 if CM is performed after  $O_{ij}$  and 0 otherwise.  
 $\xi_{ijm}$ : 1 if  $O_{ij}$  can be operated by machine  $m$  and 0 otherwise.  
 $x_{ijfkm}$ : 1 if  $O_{ij}$  is processed in the  $k$ th position on machine  $m$  in factory  $f$  and 0 otherwise.

Hence, we can establish the following mixed-integer programming model to describe optimization objectives and constraints:

$$\min f_1 = \max(E_{ij} + P'_{ij}) \quad \forall i, j \quad (1)$$

$$\min f_2 = c \left( \sum_{i,j} \eta_{ij} t_{ij} + t_{CM} \sum_{i,j} q_{ij} \right) \quad (2)$$

$$\min f_3 = \mu_1 \sum_{i,j} P'_{ij} + \mu_2 \frac{f_2}{c} + \mu_3 \sum_{i,j} I_{ij} \quad (3)$$

s.t.

$$P'_{ij} = \sum_{f,m,k} P_{ijm} x_{ijfkm} + da_{ij} \quad \forall i, j \quad (4)$$

$$t_{ij} = \min \left( t_{CM} \frac{a_{ij} + P'_{ij}}{T}, t_{CM} \right) \quad \forall i, j \quad (5)$$

$$E_{ij} \geq E_{i,j-1} + P'_{i,j-1} \quad \forall i, j \geq 2 \quad (6)$$

$$E_{ij} \geq E_{gh} + P'_{gh} + t_{gh}(\eta_{gh} + q_{gh}) - L(2 - x_{ghfm,k-1} - x_{ijfkm}) \quad \forall i, j, g, h, f, m, k \geq 2 \quad (7)$$

$$I_{ij} = E_{ij} - \left[ E_{gh} + P'_{gh} + t_{gh}(\eta_{gh} + q_{gh}) \right], \quad \text{if } x_{ghfm,k-1} + x_{ijfkm} = 2 \quad \forall i, j, g, h, f, m, k \geq 2 \quad (8)$$

$$a_{ij} \geq a_{gh} + P'_{gh} - L(2 - x_{ghfm,k-1} - x_{ijfkm} + \eta_{gh} + q_{gh}) \quad \forall i, j, g, h, f, m, k \geq 2 \quad (9)$$

$$q_{ij} \leq 1 + \frac{1}{L}(a_{ij} + P'_{ij} - T) + L(1 - x_{ijfkm}) \quad \forall i, j, f, m, k \quad (10)$$

$$q_{ij} \geq \frac{1}{L}(a_{ij} + P'_{ij} - T) - L(1 - x_{ijfkm})$$

$$\forall i, j, f, m, k \quad (11)$$

$$\eta_{ij} + q_{ij} \leq 1 \quad \forall i, j \quad (12)$$

$$\sum_{f,k} x_{ijfkm} \leq \xi_{ijm} \quad \forall i, j, m \quad (13)$$

$$\sum_{f,m,k} x_{ijfkm} = 1 \quad \forall i, j \quad (14)$$

$$\sum_{i,j} x_{ijfkm} \leq 1 \quad \forall f, m, k \quad (15)$$

$$\sum_{i,j} x_{ijfkm} \leq \sum_{i,j} x_{ijf,m,k-1} \quad \forall f, m, k \geq 2 \quad (16)$$

$$\sum_{m,k} x_{ijfkm} = \sum_{m',k'} x_{ihfjm'k'} \quad \forall i, j, h, f. \quad (17)$$

Equations (1)–(3) represent three conflicting optimization objectives, i.e., the makespan, total maintenance cost, and total energy consumption. Equation (4) specifies the relationship between actual processing time and normal processing time of each operation under deterioration effects. Equation (5) calculates the duration of PM at machine's age  $a_{ij}$ . Equation (6) ensures the sequencing between different operations for the same order, and (7) ensures the sequencing between tasks on the same machine. There may be a certain amount of idle time between two consecutively processed operations on a machine, as shown in (8). Equation (9) represents the update of the machine's age without any maintenance prior to  $O_{ij}$ . Equations (10) and (11) give the conditions for determining whether CM is executed or not. Equation (12) ensures that only one maintenance activity (either PM or CM) can be carried out behind  $O_{ij}$  at most. Equations (13)–(16) give some restrictions of the decision variable  $x_{ijfkm}$  to ensure the uniqueness of the assignment of an operation to a processing position of an available machine in a factory. Equation (17) ensures that all operations of an order are assigned to the same factory.

To validate the accuracy of our proposed model, we first linearize it in the supplementary document. Subsequently, we utilize the IBM ILOG CPLEX 22.1 solver to tackle a small-scale instance. Besides the substantial constraints and decision variables, our experimental results reveal that the DFJSP/PM exhibits the characteristic of multimodal optimization. This complexity poses a challenge for the CPLEX solver to efficiently explore optimal solutions for each individual single-objective optimization within the given time constraints. To overcome these obstacles, we introduce a customized MOEA named LBPEA in the next section.

#### IV. OUR APPROACH: LBPEA

##### A. Framework of LBPEA

We tailor an LBPEA<sup>1</sup> that considers the Pareto dominance-based and decomposition-based frameworks to solve the investigated DFJSP/PM. The framework of our LBPEA is presented in Algorithm 1. First, a three-level encoding mechanism is designed to produce the initial population at random, followed by a neighborhood search heuristic to

<sup>1</sup>Code link: <https://github.com/Rodnelps/LBPEA-code>.



**Algorithm 1: Framework of LBPEA**


---

```

1 Input: Total number of fitness evaluations ( $TNFEs$ ),
   population size ( $ps$ ), crossover rate ( $P_c$ ), mutation rate
   ( $P_m$ ), threshold for LS ( $\Delta$ ), greedy factor ( $\varepsilon$ ), discount
   factor ( $\gamma$ ), learning rate ( $\alpha$ )
2 Output: Non-dominated solution set  $\Omega$ 
3  $NFEs \leftarrow 0$ 
4  $\mathcal{P}_0 \leftarrow \text{Initialization}(ps)$  (Section IV-B)
5  $NFEs \leftarrow NFEs + ps$ ,  $PF_1 \leftarrow \emptyset$ ,  $\Omega \leftarrow \emptyset$ ,  $t \leftarrow 1$ 
6  $Q(6, 5) \leftarrow 0$ ,  $A \leftarrow \{LS_1, LS_2, LS_3, LS_4, LS_5\}$ 
7 while  $NFEs \leq TNFEs$  do
8    $\mathcal{P}_t \leftarrow \text{Tournament selection}(\mathcal{P}_t, ps)$ 
9   for  $i_1 = 1$  to  $ps$  do
10     $\mathcal{R}_t(i_1) \leftarrow \text{Evolution operator}(\mathcal{P}_t(i_1), P_c, P_m)$ 
      (Section IV-C)
11     $NFEs \leftarrow NFEs + 2$ 
12  end
13   $\mathcal{P}_t \leftarrow \mathcal{P}_t \cup \mathcal{R}_t$ 
14   $\mathcal{P}_{t+1} \leftarrow \text{Environmental Selection}(\mathcal{P}_t)$  (Section IV-D)
15   $\mathcal{P}_{t+1} \leftarrow \text{Idle time reduction strategy}(\mathcal{P}_{t+1})$ 
      (Section IV-E)
16   $NFEs \leftarrow NFEs + ps$ ,  $\Omega' \leftarrow \Omega$ 
17   $PF_{t+1} \leftarrow \text{Fast non-dominated sort}(\mathcal{P}_{t+1})$ 
18   $\Omega \leftarrow \Omega \cup PF_{t+1}$ ,  $\Omega \leftarrow \text{Delete duplicates}(\Omega)$ 
19   $\Omega \leftarrow \text{Fast non-dominated sort}(\Omega)$ 
20  if  $|\Omega' \cap \Omega|/|\Omega'| > \Delta$  then
21     $\Omega, Q \leftarrow \text{Learning-assisted adaptive local}$ 
       $\text{search}(\Omega, Q, A, \varepsilon, \gamma, \alpha)$  (Section IV-F)
22     $NFEs \leftarrow NFEs + 2 |\Omega|$ 
23     $\Omega \leftarrow \text{Delete duplicates}(\Omega)$ 
24     $\Omega \leftarrow \text{Fast non-dominated sort}(\Omega)$ 
25  end
26   $t \leftarrow t + 1$ 
27 end

```

---

increase the population's quality by modifying the codes. Then, a mating pool is created through a binary tournament and evolutionary operators including crossover and mutation are used to generate offspring. Afterward, the parents and offspring are merged, and the population is randomly divided into two subpopulations for respective environmental selection. In this way, diversity and convergence can be accommodated.

Due to the complexity of the DFJSP/PM, the above evolutionary process can easily bring the algorithm to a local optimum. To cope with this shortcoming, we develop two knowledge-driven improvement strategies. The first one is an idle time reduction strategy for improving the whole population's quality after environmental selection. The second one is a learning-assisted adaptive LS strategy to balance the computational resources between exploration and exploitation as well as adjust the neighborhood structure effectively based on the characteristic of the solution. The complexity analysis of the developed LBPEA can be found in the supplementary document.

**Algorithm 2: OM-Level Encoding**


---

```

1 Input: OS, MS, FA
2 Output: OM
3  $OM \leftarrow OS$ ,  $\beta \sim [0, \text{length}(OM)]$ 
4 if  $\beta \neq 0$  then
5    $N_{f \times m} \leftarrow 0$ 
6   Evaluate the load  $L_{f \times m}$  of machine  $m$  in factory  $f$ 
7    $P_{f \times m} \leftarrow L_{f \times m} / \sum_{f,m} L_{fm}$ ,  $P_{1 \times fm} \leftarrow P_{f \times m}$ 
8   for  $i_2 = 1$  to  $fm$  do
9      $P_{1i_2} \leftarrow \sum_{i'=1}^{i_2} P_{1i'}$ 
10  end
11   $\mathcal{P}(f, m) \leftarrow P_{1, (f-1)|M|+m}$ ,  $\mathcal{T}_{fm} \leftarrow \lceil \beta \mathcal{P}(f, m) \rceil$ 
12  for  $i_3 = 1$  to  $\beta$  do
13     $f^*, m^* \leftarrow \arg \min_{f,m} \{\mathcal{P}(f, m) | \mathcal{P}(f, m) \geq \text{rand}()\}$ 
14    if  $N_{f^*, m^*} > \mathcal{T}_{f^*, m^*}$  then
15       $L_{f^*, m^*} \leftarrow 0$ , update  $\mathbf{P}$ ,  $\mathcal{P}_{fm}$ ,  $\mathcal{T}_{fm}$ 
16    else
17       $N_{f^*, m^*} \leftarrow N_{f^*, m^*} + 1$ 
18      Randomly select an operation  $\mathcal{O}$  that is
        assigned to machine  $m^*$  in factory  $f^*$ 
19      Find the position  $p$  of  $\mathcal{O}$  in OM
20      if  $OM(p+1) = 0 \vee OM(p) = 0$  then
21        Skip to 18
22      else
23        Insert 0 after the position  $p$  in OM
24      end
25    end
26  end
27 end

```

---

**B. Initialization**

A natural idea for the representation of the solution of a DFJSP is a three-level encoding consisting of the operation sequence (OS), MS, and FA, as described in [17]. We add maintenance information to the OS level using Algorithm 2 to conform to the characteristics of the studied DFJSP/PM, which is called the operations and maintenance (OM) level in this article. At the OM level, a value of 0 after an operation means that a PM is to be performed. The number and location of PMs are unknown and need to be optimized during the evolutionary process. Algorithm 2 aims to determine a random total number  $\beta$  of PMs and balance the distribution of PMs based on the load capacity of the machines. Specifically, as the load increases, the probability of a machine being assigned a PM also increases. However, the number  $N_{fm}$  of PMs assigned to machine  $m$  in factory  $f$  should not exceed the threshold  $\mathcal{T}_{fm}$ .

An illustration of the proposed three-level codes, including OM, MS, and FA, is given in Fig. 3. To evaluate the objective functions  $f_1, f_2, f_3$ , it is necessary to traverse each operation code at the OM level from left to right. By determining the assigned factory and machine of each operation based on the FA and MS levels, we can update the start time and completion time of each operation. Besides, if there is a maintenance code (i.e., 0) after the operation at the OM level, a PM needs to be performed; Otherwise, we also need to determine whether the

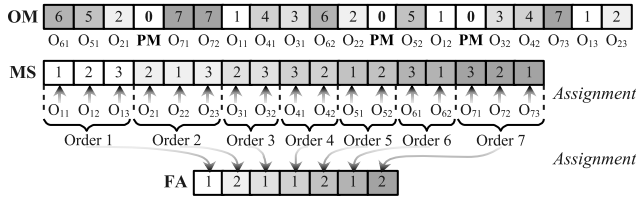


Fig. 3. Illustration of the three-level encoding.

**Algorithm 3: Neighborhood Search Heuristic**


---

```

1 Input: OM, MS, FA
2 Output: OM,  $f_1, f_2, f_3$ 
3  $p \leftarrow 1, C_{1 \times |N|} \leftarrow \mathbf{0}$  % Count the number of operations
   that have been decoded for each order.
4 while  $p \leq \text{length}(\text{OM})$  do
5    $i \leftarrow \text{OM}(p)$ , identify  $j, f, m$ 
6   Calculate  $E_{ij}$  based on (6)-(7)
7   if  $p \neq \text{length}(\text{OM})$  then
8     for  $i_4 = p + 1$  to  $\text{length}(\text{OM})$  do
9        $i' \leftarrow \text{OM}(i_4)$ 
10      if  $i' \neq 0$  then
11        Identify  $j', f', m'$ 
12        if  $f = f' \wedge m = m' \wedge j' - C_{1i'} = 1$  then
13          Calculate  $E_{i'j'}$  based on (6)-(7)
14        end
15      end
16    end
17    Determine the optimal operation  $O_{i^*}$  and its
      position  $p^*$  in OM based on Steps I-IV
18    if  $p^* < \text{length}(\text{OM}) \wedge \text{OM}(p^* + 1) = 0$  then
19       $p \leftarrow p + 2$ 
20    else
21       $p \leftarrow p + 1$ 
22    end
23    Update OM, let  $C_{1i^*} \leftarrow C_{1i^*} + 1$ 
24  end
25  Update  $I_{ij}, a_{ij}, P'_{ij}, \eta_{ij}, q_{ij}$ 
26 end
27 Calculate  $f_1, f_2, f_3$ 

```

---

machine's age reaches  $T$  to decide whether to execute a CM. The decoded scheduling sequence is shown in Fig. 1. Using the encoding and decoding methods described above, an initial population can be generated at random.

To enhance the quality of the initial population, we designed Algorithm 3 to perform a large-scale neighborhood search for the OM level, aiming to optimize the objective functions by reducing idle time. Based on the original decoding process, we propose a comparison mechanism between the current operation being considered for decoding and each subsequent operation to be decoded with the same machine and FAs. The optimal operation code determined by the following four steps will be moved before the current operation code, which is treated as a neighborhood adjustment. It should be noted that if there is a maintenance code after the optimal operation code, the two codes need to be moved together.

*Step I:* Find the operation  $O_1$  with the earliest start time. If more than one operation is found, go to step II; otherwise,  $O_1$  is the optimal operation code.

*Step II:* Find the order  $O_2$  with the highest number of remaining operations. If more than one operation is found, go to step III; otherwise,  $O_2$  is the optimal operation code.

*Step III:* Find the operation  $O_3$  with the minimal normal processing time. If more than one operation is found, go to step IV; otherwise,  $O_3$  is the optimal operation code.

*Step IV:* Find the operation  $O_4$  with a more advanced coding position, which is noted as the optimal operation code.

**C. Evolutionary Operators**

In this section, a binary tournament approach is used to construct a mating pool from which two individuals are randomly selected for crossover and mutation. The crossover and mutation rates are  $P_c$  and  $P_m$ , respectively. The specific description of the evolutionary operators is given below. In this way, the offspring population can be obtained.

*Crossover Operators:* The uniform crossover (UX) [19] is applied to the MS and FA levels. For the OM level, the encoding length may be different due to the varying number of PMs. For this reason, we first temporarily delete all 0 values in the OM, perform the precedence operation crossover (POX) [19] only for operation codes (i.e., OS), and then fill in the 0 values for the newly generated OS with reference to the insertion positions of PMs in the OM.

*Mutation Operators:* First, two different factory codes are randomly selected from the FS and swapped positions to adjust the distribution of orders among the factories. Next, a machine code is randomly selected from the MS and mutated to one of the other available machines for the corresponding operation. Finally, all 0 values in the OM are removed and two operation codes are randomly chosen and exchanged, followed by regenerating the positions of PMs using Algorithm 2 to update the OM.

**D. Environmental Selection**

After merging the parent and offspring populations, a population with the size of  $2ps$  can be obtained. It will be shuffled and evenly distributed to the two subpopulations, including *Population1* and *Population2*. Two environmental selection strategies driven by convergence and diversity are applied to *Population1* and *Population2*, respectively [22].

To be specific, solutions in *Population1* are divided into different nondominated levels using a fast nondominated sorting approach [34]. Based on the dominance relation and crowding distance of the solutions,  $ps/2$  solutions are selected. Regarding the selection of solutions in *Population2*, we first generate  $ps/2$  weight vectors uniformly based on the distribution of solutions, and the objective space is immediately divided into  $ps/2$  regions that do not overlap each other. The solutions in each region are ranked in the ascending order of the weighted objective values, and the most highly ranked solution is selected to enter into the next generation [21]. When there is no solution in a region, a solution that has not been selected in other regions is randomly chosen. Eventually, the

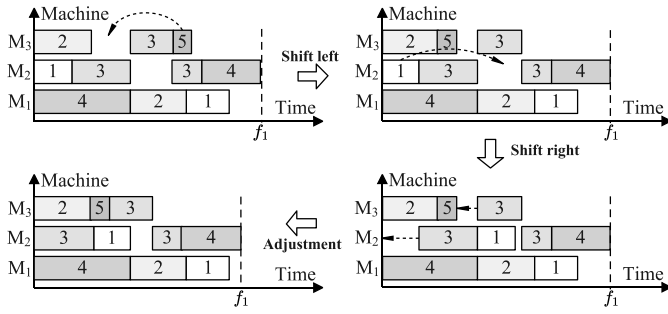


Fig. 4. Illustration of the full active schedule.

next generation population with the size of  $ps$  is obtained by combining the solutions selected from *Population1* and *Population2*.

#### E. Idle Time Reduction Strategy

In the optimization objectives, both  $f_1$  and  $f_3$  are affected by idle time. Designing an effective idle time reduction strategy can optimize  $f_1$  and  $f_3$ . The developed Algorithm 3 in the population initialization can greatly shorten idle time, but it is not suitable to be employed in the evolutionary process because the large-scale neighborhood search occupies excessive computational resources. In contrast, the full active schedule that is commonly used in FJSP-related research [17] can reduce idle time while saving computing time. Fig. 4 illustrates the full active schedule without considering deterioration and maintenance. There are two main phases, i.e., moving the operation to the left and right to the earliest idle time slot. It is evident that the key to adjusting the position of an operation is that the normal processing time of the operation is shorter than a certain idle time slot.

Accordingly, when considering deterioration and maintenance, a natural idea is to decide whether an operation can be moved or not by comparing the actual processing time of the operation with the idle time. In this case, the requirements for adjusting the position of the operation become more stringent. When the normal processing time of an operation is less than a certain idle time while the actual processing time of the operation is greater than the idle time, the operation cannot be inserted into the idle time slot. This may result in a large amount of wasted idle time.

To this end, an idle time reduction strategy based on the decomposition idea is proposed, in which the update process of the OM level is illustrated in Fig. 5. First, all 0 values in the OM are temporarily deleted, and the full active schedule illustrated in Fig. 4 is executed in each factory. Then, the deleted 0 values are filled in after the corresponding operation codes, and the decoding process presented in Section IV-B is conducted.

#### F. Learning-Assisted Adaptive LS Strategy

In complex combinatorial optimization problems, solutions generated solely by using evolutionary operators for global search are usually simple to find and only reach local optima. Incorporating suitable LS techniques can aid in adjusting the

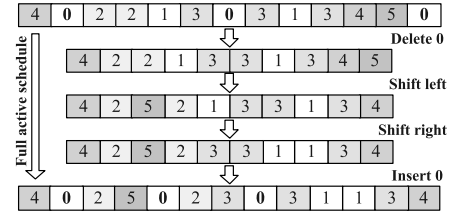


Fig. 5. Update of OM in the idle time reduction strategy.

---

#### Algorithm 4: Learning-Assisted Adaptive LS

---

```

1 Input:  $\Omega, Q, A, \varepsilon, \gamma, \alpha$ 
2 Output:  $\Omega, Q$ 
3  $\Omega^\circ \leftarrow \emptyset$ 
4 for  $i_5 = 1$  to  $|\Omega|$  do
5   // Stage one
6   Evaluate the state  $\sigma$  of the solution  $\Omega(i_5)$ 
7   if  $\text{rand}() > \varepsilon \vee Q(\sigma, :) = \mathbf{0}$  then
8      $a' \leftarrow \text{random choice}(A)$ 
9   else
10     $a' \leftarrow \arg \max_{a'' \in A} Q(\sigma, a'')$ 
11  end
12  Execute the local search corresponding to  $a'$ 
13  if  $\text{new solution} < \Omega(i_5)$  then
14     $\Omega(i_5) \leftarrow \text{new solution}$ 
15  else if  $\text{new solution} \neq \Omega(i_5)$  then
16     $\Omega^\circ \leftarrow \Omega^\circ \cup \text{new solution}$ 
17  end
18  Update the reward  $r$  and new state  $\sigma'$ 
19   $Q(\sigma, a') \leftarrow (1 - \alpha)Q(\sigma, a') + \alpha[r +$ 
20     $\gamma \max_{a'' \in A} Q(\sigma', a'')]$ 
21  // Stage two
22  Execute the maintenance cost saving strategy for
23   $\Omega(i_5)$ , and update  $\Omega$  or  $\Omega^\circ$  based on steps 13-17
24 end
25  $\Omega \leftarrow \Omega \cup \Omega^\circ$ , Delete duplicates( $\Omega$ )
26  $\Omega \leftarrow \text{Fast nondominated sort}(\Omega)$ 

```

---

neighborhood structure of solutions and exploring a more promising solution space. Furthermore, it is necessary to balance the allocation of computational resources between local and global search. For this reason, this work proposes a learning-assisted adaptive LS strategy, which is triggered when the proportion of solutions in the  $t$ -th generation Pareto solution set that are inherited to the  $t+1$ th generation is larger than a set threshold  $\Delta$ . As shown in Algorithm 4, each solution in  $\Omega$  undergoes a two-stage LS process.

At the first stage, we propose five LS operators, including  $LS_1, LS_2, LS_3, LS_4$ , and  $LS_5$  to adjust the operations' sequence or the assignment of operations across different factories and machines.

- 1)  $LS_1$ : All 0 values in the OM-level encoding are temporarily deleted, and the classical  $N6$  structure is introduced to adjust the operations' sequence on the critical path [17]. Subsequently, the deleted 0 values are filled in after the corresponding operation codes.

- 2)  $LS_2$ : An order is randomly selected from the factory with the longest completion time and assigned to other factories.
- 3)  $LS_3$ : An operation is randomly selected from the critical path and assigned to other available machines.
- 4)  $LS_4$ : Randomly select an operation from each order and disrupt their positions in the OM-level encoding.
- 5)  $LS_5$ : Randomly select some operations and assign them to the machine with the shortest processing time.

Considering the finiteness of the computation time, it is inappropriate to perform all the LS operators for each solution. Although the random selection method is simple, it may result in the chosen LS operator not being more effective due to ignoring the feature information of the solution.  $Q$ -learning as a model-free RL method has been elucidated the ability to adaptively select parameters or operators [19], [24], [35]. Another important advantage is that it enables online learning without increasing the algorithm's complexity. Consequently, we introduce the  $Q$ -learning algorithm to select LS operators based on the characteristic of multiobjective optimization, which is distinct from previous research. The state, action and reward are defined as follows.

The state is measured by the ordering of the normalized objective values ( $f_1^*, f_2^*, f_3^*$ ) in different coordinate dimensions. In the studied three-objective optimization, there are six permutations, including  $f_1^* \geq f_2^* \geq f_3^*$ ,  $f_1^* \geq f_3^* \geq f_2^*$ ,  $f_2^* \geq f_1^* \geq f_3^*$ ,  $f_2^* \geq f_3^* \geq f_1^*$ ,  $f_3^* \geq f_1^* \geq f_2^*$ , and  $f_3^* \geq f_2^* \geq f_1^*$ , which constitute the state space. A state is chosen randomly when the occurrence of an equality relationship causes some states to be the same. In the action space, there are five actions, including  $LS_1$ ,  $LS_2$ ,  $LS_3$ ,  $LS_4$ , and  $LS_5$ . After executing a specific action, a new solution ( $f_1^o, f_2^o, f_3^o$ ) will be generated and the reward will be updated according to

$$r = -1, \begin{cases} \text{if } \forall w \in 1, 2, 3, f_w^* - f_w^o \leq 0 \\ \sum_{w=1}^3 r_w \max\{f_w^* - f_w^o, 0\}, \text{ otherwise} \end{cases} \quad (18)$$

where  $(r_1, r_2, r_3)$  is an arrangement of  $(0.1, 0.2, 0.3)$ , determined by the descending order of  $f_1^*, f_2^*, f_3^*$ .

The setting of the reward is to encourage the normalized solutions to preferentially move in the dimension of the farthest deviation from the ideal point  $(0, 0, 0)$ . Therefore, we give the maximum reward weight of 0.3 if there is an improvement in the dimension of the farthest deviation after the execution of LS, and give the reward weights of 0.2 and 0.1 if there is an improvement in the last two dimensions, respectively. Finally, we calculate the weighted sum of the magnitude of improvement in each objective dimension as the cumulative reward. If there is no improvement in each dimension, a penalty of -1 is given. An illustration of reward updates using different LS operators is shown in Fig. 6.

The magnitude relationship of coordinate values of the black point  $(0.35, 0.85, 0.7)$  is  $0.85 > 0.7 > 0.35$ , and thus the corresponding reward weights for the improvement in the  $x$ ,  $y$ , and  $z$  dimensions are 0.1, 0.3, and 0.2, respectively. Assuming that the black point is guided by  $LS_1$ ,  $LS_2$ ,  $LS_3$ ,  $LS_4$ , and  $LS_5$  to move to positions  $(0.45, 0.9, 0.8)$ ,  $(0.75, 0.95, 0.3)$ ,  $(0.65, 0.55, 0.6)$ ,  $(0.3, 0.75, 0.65)$ , and  $(0.15, 0.8, 0.8)$ , the

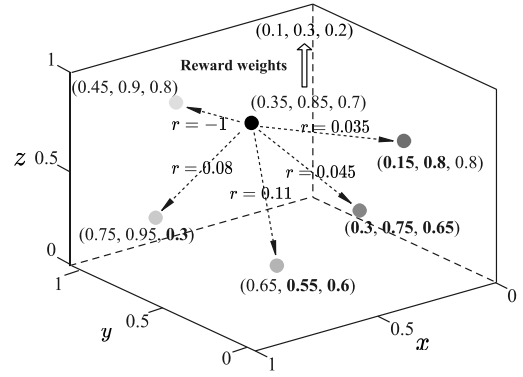


Fig. 6. Illustration of reward updates using different LS operators.

corresponding rewards can be calculated as  $-1$ ,  $0.08$ ,  $0.11$ ,  $0.045$ , and  $0.035$  based on (18).

The next is the second stage of the learning-assisted adaptive LS strategy, primarily adjusting the number and placement of maintenance decisions, which is treated as the maintenance adjustment strategy. The first step is to determine whether PM is assigned after the last operation of each machine, and if so, the corresponding maintenance code is removed from the OM-level encoding. Second, it is necessary to determine whether CM is required on each machine. If CM occurs after a certain operation, it means that PM is not timely, and we choose to execute PM before this operation to improve the machine's status in advance while reducing the maintenance cost; Otherwise, the current number of PM may be moderate or excessive, and thus we need to adjust the number or position of PMs based on the following judgment. If the number of PMs assigned to a machine is greater than half of the number of operations, we randomly decrease a PM or randomly move a PM after another operation with a greater probability, and randomly increase a PM with a smaller probability; Otherwise, one of the following steps is taken with equal probability: randomly adding a PM, randomly subtracting a PM, or keeping the number and position of PMs constant.

## V. EXPERIMENTAL RESULTS

In this section, we first perform orthogonal tests on the key parameters of LBPEA, and then compare LBPEA with its peers and five state-of-the-art MOEAs on 20 extended benchmarks. All algorithms were coded in MATLAB R2023a and were run ten times independently under each benchmark on a personal computer with Intel Core i7-14700KF and 64.0-GB RAM.

### A. Test Instances

Due to the inadequacy of DFJSP benchmarks, we selected the FJSP benchmarks MK01-10 and DP01-10 with reference to [17]. To fit the characteristics of the proposed DFJSP/PM, we added the number of factories in each benchmark, where three homogeneous factories were set in benchmarks MK07-10 and DP09-10, and two homogeneous factories were considered in other benchmarks. Furthermore, we assumed that each machine has the same deterioration rate  $d$  of 0.1.



The threshold  $T$  for triggering a CM was set to 60, and the duration  $t_{CM}$  as well as maintenance cost  $c$  per unit time of a CM were 5 and 10. The energy consumptions  $\mu_1, \mu_2$ , and  $\mu_3$  per unit time in production, maintenance, and standby modes were set to 8, 2, and 1, respectively. Following the above, a total of 20 extended benchmarks were obtained, denoted as EMk01-10 and EDP01-10 in this article.

### B. Evaluation Metrics

The overall performance of MOEAs is typically evaluated in terms of both convergence and diversity, and the most frequently used evaluation metrics are inverted generational distance (IGD) and hypervolume (HV), respectively [36], [37]. To eliminate the difference in magnitude among multiple objectives  $f_w(x)$ , ( $w \in \mathbb{N}^+$ ,  $x \in \Omega$ ), the nondominated solutions are usually normalized before calculating the above evaluation metrics, as presented in

$$f_w^*(x) = \frac{f_w(x) - f_w^{\min}}{f_w^{\max} - f_w^{\min}} \quad (19)$$

where  $f_w^{\min}$  and  $f_w^{\max}$  are the minimum and maximum values of  $f_w(x)$  among the nondominated solutions, and  $f_w^*(x)$  represents the normalized solutions.

The IGD metric reflects the average distance between a set  $\Omega$  of nondominated solutions and the true Pareto front PF, as shown in (20). A smaller IGD value means that the nondominated solutions are closer to the true Pareto front and the algorithm converges better. Since the true Pareto front of the proposed DFJSP/PM is unknown in advance, we ran each competing algorithm ten times on each benchmark separately and merged all the nondominated solutions to approximate the Pareto front

$$IGD = \frac{1}{|PF|} \sum_{\omega^2 \in PF} \min_{\omega^1 \in \Omega} \|\omega^1 - \omega^2\|. \quad (20)$$

The HV metric is the volume of the hypercube enclosed between a set  $\Omega$  of nondominated solutions and the corresponding nadir point  $\pi$ , as calculated in (21). Unlike the IGD metric, this metric does not depend on the true Pareto front. The closer the HV value is to 1, the better the MOEA performs

$$HV = \lambda_w(\cup_{x \in \Omega} [x, \pi]) \quad (21)$$

where  $\lambda_w$  denotes the  $w$ -dimensional Lebesgue measure.

To ensure the accuracy of calculating the above metrics, we called the source codes for the IGD and HV metrics provided in the PlatEMO [38] after running all the algorithms to obtain nondominated solutions.

### C. Parameter Settings

Through a series of preliminary experiments, we set the total number of fitness evaluations of 70 000 as the termination condition to ensure that the algorithm reaches convergence under each benchmark. In addition, there are seven key parameters that may influence the overall performance of the developed LBPEA, i.e., the population size ( $ps$ ), threshold for triggering LS ( $\Delta$ ), crossover rate ( $P_c$ ), mutation rate ( $P_m$ ), greedy factor ( $\varepsilon$ ), discount factor ( $\gamma$ ), and learning rate ( $\alpha$ ). We

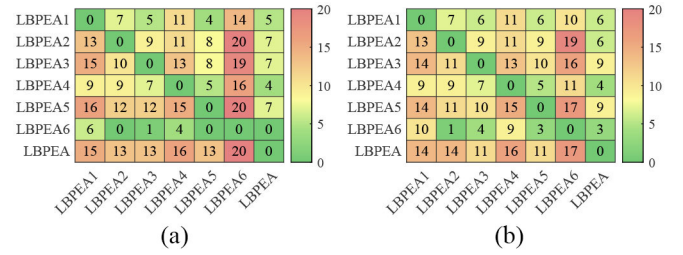


Fig. 7. Number of wins of each variant in pairwise comparisons against other variants across 20 benchmarks. (a) Under the IGD metric. (b) Under the HV metric.

adopted the widely used Taguchi-based design of experiments to avoid enumerating all possible parameters' combinations while selecting a promising configuration of key parameters [17], [19], [22]. We assumed that each key parameter has four levels:  $ps \in \{40, 60, 80, 100\}$ ,  $\Delta \in \{0.8, 0.85, 0.9, 0.95\}$ ,  $P_c \in \{0.7, 0.8, 0.9, 1\}$ ,  $P_m \in \{0.1, 0.15, 0.2, 0.25\}$ ,  $\varepsilon \in \{0.6, 0.7, 0.8, 0.9\}$ ,  $\gamma \in \{0.6, 0.7, 0.8, 0.9\}$ , and  $\alpha \in \{0.1, 0.2, 0.3, 0.4\}$ . An orthogonal array was generated, and benchmarks EMk02 and EMk07 were selected as test instances. To reduce the variance coming from the algorithm's randomness and attain reliable results, we conducted ten independent experiments under each set of parameters and calculated the average IGD value to evaluate the performance. Based on the detailed analysis of the factor level trend of key parameters in the supplementary document, we finally set the following combination:  $ps = 100$ ,  $\Delta = 0.9$ ,  $P_c = 1$ ,  $P_m = 0.25$ ,  $\varepsilon = 0.6$ ,  $\gamma = 0.9$ , and  $\alpha = 0.2$ .

### D. Evaluation of Each Improvement in LBPEA

To assess the effectiveness of each improvement in LBPEA, we designed six variants of LBPEA, denoted as LBPEA1, LBPEA2, LBPEA3, LBPEA4, LBPEA5, and LBPEA6. In LBPEA1 and LBPEA2, convergence-driven and diversity-driven environmental selection strategies are, respectively, used to replace the bi-population environmental selection strategy. LBPEA3 removes the neighborhood search heuristic in the initialization process. LBPEA4 deletes the idle time reduction strategy. LBPEA5 randomly selects LS operators to replace the  $Q$ -learning process. LBPEA6 removes the maintenance adjustment strategy from LBPEA. We evaluated LBPEA as well as its six variants on 20 benchmarks in terms of IGD and HV metrics. For each benchmark, all algorithms were run independently ten times. The mean value and standard deviation of IGD and HV values are presented in the supplementary document. Here, Fig. 7 intuitively shows the number of wins of each variant in pairwise comparisons against other variants. The main observations from comparative results are summarized as follows.

First, LBPEA has a clear advantage over LBPEA1 and LBPEA2, winning 15, 13 times under the IGD metric, and 14, 14 times under the HV metric. This demonstrates the positive effect using the bi-population environmental selection strategy on the convergence and diversity of LBPEA. Second, when comparing LBPEA with LBPEA3, we can find that the neighborhood search heuristic make LBPEA play more

TABLE II  
RANKING RESULTS OF LBPEA AND ITS SIX VARIANTS BY THE  
FRIEDMAN TEST AT A 0.05 SIGNIFICANT LEVEL, WHERE THE  
BEST AVERAGE RANKING IS IN BOLD

| MOEAs  | IGD          |           | HV           |           |
|--------|--------------|-----------|--------------|-----------|
|        | rank         | p-value   | rank         | p-value   |
| LBPEA1 | 4.700        |           | 3.300        |           |
| LBPEA2 | 3.575        |           | 4.350        |           |
| LBPEA3 | 3.375        |           | 4.650        |           |
| LBPEA4 | 4.500        | 2.683E-08 | 3.250        | 3.563E-04 |
| LBPEA5 | 2.900        |           | 4.800        |           |
| LBPEA6 | 6.450        |           | 2.500        |           |
| LBPEA  | <b>2.500</b> |           | <b>5.150</b> |           |

consistently than LBPEA3 on different benchmarks. Third, LBPEA outperforms LBPEA4 in 16 out of 20 comparisons under the IGV and HV metrics. This indicates that performing the idle time strategy is crucial to enhance the algorithm's performance. Besides, LBPEA is slightly better than LBPEA5, winning 13 and 11 times, respectively, under the IGD and HV metrics. Meanwhile, LBPEA shows an absolute advantage over LBPEA6, winning 20 and 17 times, respectively, under the IGD and HV metrics. This suggests that the learning-assisted LS strategy can improve the stability and convergence speed of the algorithm.

Table II presents the ranking results of LBPEA and its six variants using the Friedman test at a 0.05 significant level. It can be seen that LBPEA has the best average ranking of 2.500 under the IGD metric and 5.150 under the HV metric. In contrast, LBPEA6 obtains the worst average ranking of 6.450 under the IGD metric and 2.500 under the HV metric. This demonstrates that each improvement in the developed LBPEA is effective, with the maintenance adjustment strategy playing the most crucial role. Meanwhile, the  $p$ -values under the IGD and HV metrics are far less than 0.05, indicating significant differences between LBPEA and its six variants.

#### E. Comparisons With Other MOEAs

The effectiveness of LBPEA in solving the multiobjective DFJSP/PM has been initially verified in Section V-D. To further evaluate the performance of LBPEA, we selected five state-of-the-art MOEAs, including the SPAMA [17], LRVMA [19], BPBMO [22], TS-KEA [39], and KBEA [40] as rivals. Specific reasons for the selection are presented below.

- 1) SPAMA, TS-KEA, and KBEA were developed based on the NSGA-II [34] to solve energy-efficient bi-objective DFJSPs. In SPAMA, the surprisingly popular algorithm was used to select the promising LS operator with low selection probability. In TS-KEA, the evolutionary process is divided into two stages, aiming at efficiently approaching to the Pareto front and exploring a broader range of nondominated solutions. In KBEA, the population is categorized into two subpopulations, with each subpopulation tailored to pursue its own evolutionary strategies to search solutions from different directions.
- 2) LRVMA was proposed based on the MOEA/D [41] to tackle an energy-efficient bi-objective FJSP, in which the  $Q$ -learning algorithm was employed to adaptively determine the neighborhood's size by measuring the change



Fig. 8. Number of wins of each algorithm in pairwise comparisons against other algorithms across 20 benchmarks. (a) Under the IGD metric. (b) Under the HV metric.

TABLE III  
RANKING RESULTS OF LBPEA AND ITS FIVE COMPETITORS BY THE  
FRIEDMAN TEST AT A 0.05 SIGNIFICANT LEVEL, WHERE THE  
BEST AVERAGE RANKING IS IN BOLD

| MOEAs  | IGD          |           | HV           |           |
|--------|--------------|-----------|--------------|-----------|
|        | rank         | p-value   | rank         | p-value   |
| SPAMA  | 2.350        |           | 4.900        |           |
| LRVMA  | 3.950        |           | 2.150        |           |
| BPBMO  | 5.500        |           | 1.500        |           |
| TS-KEA | 3.150        | 3.674E-16 | 4.050        | 1.184E-17 |
| KBEA   | 5.050        |           | 2.400        |           |
| LBPEA  | <b>1.000</b> |           | <b>6.000</b> |           |

of nondominated solutions in terms of convergence and diversity.

- 3) BPBMO was designed based on the c-DPEA [21] to deal with a DFJSP considering crane transportation. The balancing crossover heuristic, cooperative environmental selection mechanism, and adaptive adjustment of the PS were proposed to balance convergence and diversity.

To enable the above five competing algorithms to solve the proposed multiobjective DFJSP/PM while ensuring a fair comparison, we adopted the same encoding and decoding mechanism, PS, and termination condition for all the algorithms. The other parameter settings and operator designs of the algorithm rivals were configured as recommended in their original literature. All algorithms were conducted ten runs under each benchmark, and the mean and standard deviation values of IGD and HV metrics were obtained as shown in the supplementary document. Here, we display the number of wins of each algorithm in pairwise comparisons against other algorithms across 20 benchmarks. From the last row of the matrix in Fig. 8(a) and (b), it is obvious that LBPEA beats SPAMA, LRVMA, BPBMO, TS-KEA, and KBEA on all benchmarks under the IGD and HV metrics. This means that the nondominated solutions obtained by LBPEA have better convergence and diversity.

To further analyze the difference between LBPEA and its competitors, ranking results of six algorithms using the Friedman test at a 0.05 significant level are given in Table III. Obviously, LBPEA obtains a better average ranking than other algorithms under the IGD and HV metrics. Meanwhile, the  $p$ -values are far less than 0.05, indicating significant differences between LBPEA and five competing algorithms.

The success of LBPEA is attributed to the synergistic effect of the following components. First, the neighborhood search heuristic can enhance the quality of the initial population.

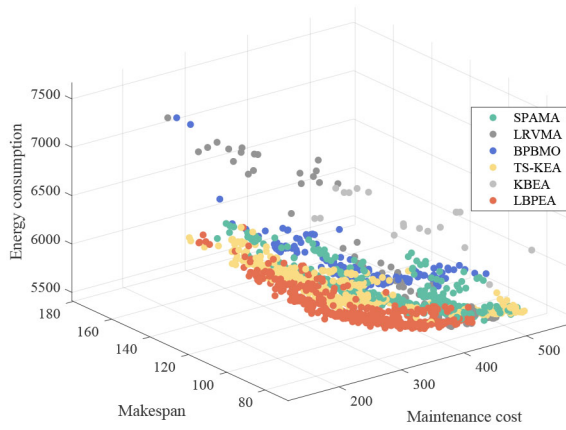


Fig. 9. Algorithms' comparison on EMk07 in terms of the Pareto front.

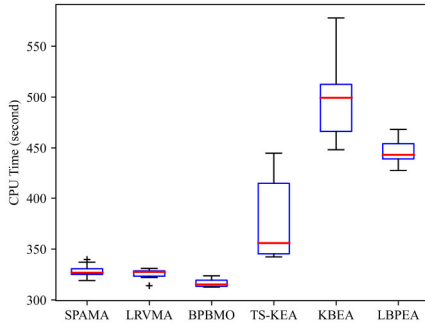


Fig. 10. Box plots of CPU time spent on EMk07 by the six algorithms.

Second, the implementation of bi-population-based environment selection can strike a balance between the convergence and diversity of nondominated solutions. Next, the idle time reduction strategy can accelerate the search in the  $f_1$  and  $f_3$  directions, while the execution of the maintenance adjustment strategy increases the exploration in the  $f_2$  direction. Besides, two-stage LS process can assist LBPEA in escaping local optima and fostering rapid convergence of nondominated solutions.

Furthermore, we visualize the Pareto fronts obtained after ten independent runs of all algorithms on EMk07, as shown in Fig. 9. It is clear that LBPEA dominates five algorithm competitors. Meanwhile, the average running time of all algorithms on EMk07 is given in Fig. 10. We can find that the running time of LBPEA is not much different from five algorithm competitors. This indicates that LBPEA can effectively provide closer approximations to the true Pareto front in solving the studied multiobjective DFJSP/PM.

## VI. CONCLUSION

This article investigated a multiobjective integrated decision model of the DFJSP and PM. Compared to a predetermined PM strategy, the proposed model covers a larger solution space and can provide more potential solutions for decision makers. This poses a high challenge to the design of the optimization algorithm. To jump out of this dilemma caused by huge maintenance decisions, we allocated PMs in the coding in

a balanced way according to the load capacity of each machine and proposed a maintenance adjustment strategy in the LS.

Another major contribution of the developed LBPEA is the selection of LS operators based on  $Q$ -learning. In this way, the solution is encouraged to move in the dimension of the farthest deviation from the ideal point, which can speed up convergence. Computational experiments demonstrate that LBPEA exhibits outstanding performance over five state-of-the-art MOEAs for solving the studied multiobjective DFJSP/PM.

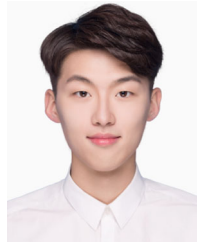
In our future research, the developed LBPEA will be further optimized in combination with actual scenarios such as furniture customization and automobile assembly. More investigations on evolutionary multiobjective optimization algorithms help to solve other large-scale combinational optimization problems. Furthermore, the definitions of state, action, and reward in the learning process can be extended and applied to higher-dimensional MOEAs, providing a generalized and effective approach to select LS strategies.

## REFERENCES

- [1] P. Chowdhury, S. K. Paul, S. Kaisar, and M. A. Moktadir, "COVID-19 pandemic related supply chain studies: A systematic review," *Transp. Res. Part-E: Logist. Transp. Rev.*, vol. 148, Apr. 2021, Art. no. 102271.
- [2] J. S. Srai et al., "Distributed manufacturing: Scope, challenges and opportunities," *Int. J. Prod. Res.*, vol. 54, no. 23, pp. 6917–6935, 2016.
- [3] N. Bagheri Rad and J. Behnamian, "Recent trends in distributed production network scheduling problem," *Artif. Intell. Rev.*, vol. 55, pp. 2945–2995, Apr. 2022.
- [4] E. Rauch, P. Dallasega, and D. T. Matt, "Sustainable production in emerging markets through distributed manufacturing systems (DMS)," *J. Clean. Prod.*, vol. 135, pp. 127–138, Nov. 2016.
- [5] Y. Fu, Y. Hou, Z. Wang, X. Wu, K. Gao, and L. Wang, "Distributed scheduling problems in intelligent manufacturing systems," *Tsinghua Sci. Technol.*, vol. 26, no. 5, pp. 625–645, Oct. 2021.
- [6] Q. Luo, Q. Deng, G. Gong, L. Zhang, W. Han, and K. Li, "An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers," *Expert Syst. Appl.*, vol. 160, Dec. 2020, Art. no. 113721.
- [7] Y. Du, J. Li, C. Luo, and L. Meng, "A hybrid estimation of distribution algorithm for distributed flexible job shop scheduling with crane transportation," *Swarm Evol. Comput.*, vol. 62, Apr. 2021, Art. no. 100861.
- [8] S. Dauzère-Pérès, J. Ding, L. Shen, and K. Tamssaouet, "The flexible job shop scheduling problem: A review," *Eur. J. Oper. Res.*, vol. 314, no. 2, pp. 409–432, 2023.
- [9] H. Wang, Q. Yan, and S. Zhang, "Integrated scheduling and flexible maintenance in deteriorating multi-state single machine system using a reinforcement learning approach," *Adv. Eng. Inform.*, vol. 49, Aug. 2021, Art. no. 101339.
- [10] M. C. Santoro and L. Junqueira, "Unrelated parallel machine scheduling models with machine availability and eligibility constraints," *Comput. Ind. Eng.*, vol. 179, May 2023, Art. no. 109219.
- [11] H. Ye, X. Wang, and K. Liu, "Adaptive preventive maintenance for flow shop scheduling with resumable processing," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 1, pp. 106–113, Jan. 2021.
- [12] Y. An, X. Chen, K. Gao, Y. Li, and L. Zhang, "Multiobjective flexible job-shop rescheduling with new job insertion and machine preventive maintenance," *IEEE Trans. Cybern.*, vol. 53, no. 5, pp. 3101–3113, May 2023.
- [13] F. T. Chan, S. Chung, L. Chan, G. Finke, and M. Tiwari, "Solving distributed FMS scheduling problems subject to maintenance: Genetic algorithms approach," *Robot. Comput.-Integr. Manuf.*, vol. 22, no. 5, pp. 493–504, Dec. 2006.
- [14] S. Chung, F. T. Chan, and H. Chan, "A modified genetic algorithm approach for scheduling of perfect maintenance in distributed production scheduling," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 1005–1014, 2009.
- [15] S. Chung, H. Lau, G. Ho, and W. Ip, "Optimization of system reliability in multi-factory production networks by maintenance approach," *Expert Syst. Appl.*, vol. 36, no. 6, pp. 10188–10196, 2009.



- [16] C.-S. Lin, I.-L. Lee, and M.-C. Wu, "Merits of using chromosome representations and shadow chromosomes in genetic algorithms for solving scheduling problems," *Robot. Comput.-Integr. Manuf.*, vol. 58, pp. 196–207, Aug. 2019.
- [17] R. Li, W. Gong, L. Wang, C. Lu, and X. Zhuang, "Surprisingly popular-based adaptive memetic algorithm for energy-efficient distributed flexible job shop scheduling," *IEEE Trans. Cybern.*, vol. 53, no. 12, pp. 8013–8023, Dec. 2023.
- [18] E. Jiang, L. Wang, and Z. Peng, "Solving energy-efficient distributed job shop scheduling via multi-objective evolutionary algorithm with decomposition," *Swarm Evol. Comput.*, vol. 58, Nov. 2020, Art. no. 100745.
- [19] R. Li, W. Gong, C. Lu, and L. Wang, "A learning-based memetic algorithm for energy-efficient flexible job-shop scheduling with type-2 fuzzy processing time," *IEEE Trans. Evol. Comput.*, vol. 27, no. 3, pp. 610–620, Jun. 2023.
- [20] H.-L. Liu, L. Chen, K. Deb, and E. D. Goodman, "Investigating the effect of imbalance between convergence and diversity in evolutionary multiobjective algorithms," *IEEE Trans. Evol. Comput.*, vol. 21, no. 3, pp. 408–425, Jun. 2017.
- [21] M. Ming, A. Trivedi, R. Wang, D. Srinivasan, and T. Zhang, "A dual-population-based evolutionary algorithm for constrained multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 25, no. 4, pp. 739–753, Aug. 2021.
- [22] J. Li, Y. Han, K. Gao, X. Xiao, and P. Duan, "Bi-population balancing multi-objective algorithm for fuzzy flexible job shop with energy and transportation," *IEEE Trans. Autom. Sci. Eng.*, early access, Aug. 7, 2023, doi: [10.1109/TASE.2023.3300922](https://doi.org/10.1109/TASE.2023.3300922).
- [23] J. Xie, X. Li, L. Gao, and L. Gui, "A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems," *J. Manuf. Syst.*, vol. 71, pp. 82–94, Dec. 2023.
- [24] Z.-Q. Zhang, F.-C. Wu, B. Qian, R. Hu, L. Wang, and H.-P. Jin, "A Q-learning-based hyper-heuristic evolutionary algorithm for the distributed flexible job-shop scheduling problem with crane transportation," *Expert Syst. Appl.*, vol. 234, Dec. 2023, Art. no. 121050.
- [25] N. Zhu, G. Gong, D. Lu, D. Huang, N. Peng, and H. Qi, "An effective reformative memetic algorithm for distributed flexible job-shop scheduling problem with order cancellation," *Expert Syst. Appl.*, vol. 237, Mar. 2024, Art. no. 121205.
- [26] L. Wang, Z. Pan, and J. Wang, "A review of reinforcement learning based intelligent optimization for manufacturing scheduling," *Complex Syst. Model. Simul.*, vol. 1, no. 4, pp. 257–270, 2021.
- [27] Z. Zhang, L. Zheng, F. Hou, and N. Li, "Semiconductor final test scheduling with Sarsa( $\lambda, k$ ) algorithm," *Eur. J. Oper. Res.*, vol. 215, no. 2, pp. 446–458, 2011.
- [28] S. Luo, L. Zhang, and Y. Fan, "Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 4, pp. 3020–3038, Oct. 2022.
- [29] Z. Zeng, X. Li, and C. Bai, "A deep reinforcement learning approach to flexible job shop scheduling," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, 2022, pp. 884–890.
- [30] I.-B. Park and J. Park, "Scalable scheduling of semiconductor packaging facilities using deep reinforcement learning," *IEEE Trans. Cybern.*, vol. 53, no. 6, pp. 3518–3531, Jun. 2023.
- [31] Z. Cao, C. Lin, M. Zhou, and R. Huang, "Scheduling semiconductor testing facility by using cuckoo search algorithm with reinforcement learning and surrogate modeling," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 2, pp. 825–837, Apr. 2019.
- [32] Y. Du, J. Li, X. Chen, P. Duan, and Q. Pan, "Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 4, pp. 1036–1050, Aug. 2023.
- [33] Q. Yan, H. Wang, and F. Wu, "Digital twin-enabled dynamic scheduling with preventive maintenance using a double-layer Q-learning algorithm," *Comput. Oper. Res.*, vol. 144, Aug. 2022, Art. no. 105823.
- [34] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [35] Q. Yan and H. Wang, "Double-layer Q-learning-based joint decision-making of dual resource-constrained aircraft assembly scheduling and flexible preventive maintenance," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 58, no. 6, pp. 4938–4952, Dec. 2022.
- [36] C. Audet, J. Bignon, D. Cartier, S. Le Digabel, and L. Salomon, "Performance indicators in multiobjective optimization," *Eur. J. Oper. Res.*, vol. 292, no. 2, pp. 397–422, 2021.
- [37] Y. Li, Q. Zhao, S. Yang, and Y. Guo, "Tailoring evolutionary algorithms to solve the multi-objective location-routing problem for biomass waste collection," *IEEE Trans. Evol. Comput.*, vol. 28, no. 2, pp. 489–500, Apr. 2024.
- [38] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [educational forum]," *IEEE Comput. Intell. Mag.*, vol. 12, no. 4, pp. 73–87, Nov. 2017.
- [39] R. Li, W. Gong, L. Wang, C. Lu, and S. Jiang, "Two-stage knowledge-driven evolutionary algorithm for distributed green flexible job shop scheduling with type-2 fuzzy processing time," *Swarm Evol. Comput.*, vol. 74, Oct. 2022, Art. no. 101139.
- [40] F. Yu, C. Lu, J. Zhou, L. Yin, and K. Wang, "A knowledge-guided bi-population evolutionary algorithm for energy-efficient scheduling of distributed flexible job shop problem," *Eng. Appl. Artif. Intell.*, vol. 128, Feb. 2024, Art. no. 107458.
- [41] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.



**Qi Yan** received the B.Sc. degree in mathematics and applied mathematics from Shenyang Normal University, Shenyang, China, in 2018. He is currently pursuing the Ph.D. degree with the College of Information Science and Engineering, Northeastern University, Shenyang.

He has published eight research papers in journals and conferences. His research focuses on production scheduling and preventive maintenance, evolutionary algorithms, reinforcement learning, and intelligent manufacturing.



**Hongfeng Wang** received the B.Sc. degree in industrial engineering and the M.Sc. and Ph.D. degrees in system engineering from Northeastern University, Shenyang, China, in 2001, 2004, and 2007, respectively.

He is currently a Professor with the Institute of System Engineering, Northeastern University. He has authored or coauthored more than 80 refereed journal papers. His current research interests include model and optimization for complex system, manufacturing system engineering, and production operation management.



**Shengxiang Yang** (Senior Member, IEEE) received the Ph.D. degree from Northeastern University, Shenyang, China, in 1999.

He is currently a Professor of Computational Intelligence and the Deputy Director of the Institute of Artificial Intelligence, School of Computer Science and Informatics, De Montfort University, Leicester, U.K. He has over 430 publications with an H-index of 73 according to Google Scholar. His current research interests include evolutionary computation, swarm intelligence, artificial neural networks, data mining and data stream mining, and relevant real-world applications.

Prof. Yang serves as an Associate Editor/Editorial Board Member for a number of international journals, such as the IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, IEEE TRANSACTIONS ON CYBERNETICS, Information Sciences, Enterprise Information Systems, and CAAI Transactions on Intelligence Technology.